

Relatório Técnico Final — HyperAutomation

1. Identificação do Projeto

Campo	Informação
Projeto	HyperAutomation — Portal Fake Soluções Digitais
Disciplina	Técnicas de Hyperautomation
Professor(a)	Moisés Levy
Turma	Turma 02 — AX Academy / IFAM
Data de entrega	23/07/2026

Integrantes da Equipe

Nome	Responsabilidade
Romulo Lira	Líder do projeto, integração dos módulos, GitFlow e versionamento
Huan Cruz De Oliveira	Modelagem BPMN e documentação visual
Gilvan Daniel da Silva	Geração da ficha de cadastro em Word (.docx)
Jannas	Extração de dados do Portal Fake (Playwright)

2. Objetivo da Automação

O objetivo deste projeto é automatizar o processo de cadastro de clientes do **Portal Fake Soluções Digitais**, uma aplicação web simulada que gera fichas de cadastro com dados aleatórios.

A automação abrange três etapas principais:

Extração de dados: Um robô (script Python com Playwright) acessa o Portal Fake, clica no botão "Novo Cadastro" e extrai automaticamente os dados gerados no formulário (nome, sobrenome, CPF, e-mail, telefone, data de nascimento e endereço).

Geração de documento: Os dados extraídos são utilizados para gerar uma ficha de cadastro formatada em documento Word (.docx), com tabela de dados, seção de documentos necessários e campo de

assinatura.

Envio por e-mail: A ficha gerada é enviada automaticamente como anexo de e-mail para um destinatário configurado, utilizando o protocolo SMTP com criptografia TLS.

O projeto demonstra na prática os conceitos de **Hyperautomation** estudados no módulo, integrando automação web (RPA), geração de documentos e comunicação automatizada em um fluxo único e orquestrado.

3. Modelagem do Processo (BPMN)

Diagrama BPMN

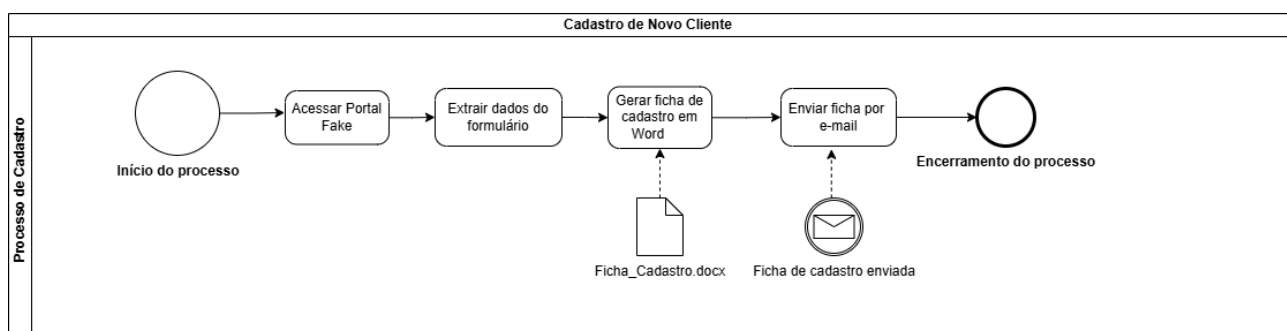


Figura 1: Diagrama BPMN do processo de cadastro automatizado no Portal Fake Soluções Digitais.

Descrição do Fluxo

O diagrama BPMN modela o processo "**Cadastro de Novo Cliente**" dentro da swimlane "**Processo de Cadastro**", com as seguintes etapas:

Início do processo — Evento de início (círculo simples) que dispara a automação quando o script `main.py` é executado.

Acessar Portal Fake — O robô abre o navegador Chromium via Playwright e navega até o Portal Fake Soluções Digitais (`portal_fake/index.html`).

Extrair dados do formulário — O robô clica no botão "Novo Cadastro" (`#btnNovo`) e captura os valores dos 7 campos do formulário: Nome, Sobrenome, CPF, E-mail, Telefone, Data de Nascimento e Endereço.

Gerar ficha de cadastro em Word — Os dados extraídos são passados para o módulo `documento_email.py`, que gera um documento `.docx` formatado com tabela de dados, lista de documentos necessários e campo de assinatura. O artefato de dados **Ficha_Cadastro.docx** é representado como objeto de dados no diagrama.

Enviar ficha por e-mail — O módulo `envio_email.py` conecta-se ao servidor SMTP configurado (via TLS), autentica com as credenciais do arquivo `.env` e envia a ficha como anexo para o destinatário. O evento intermediário de mensagem "**Ficha de cadastro enviada**" confirma o envio.

Encerramento do processo — Evento de fim (círculo com borda dupla) indicando que o fluxo foi concluído com sucesso.

4. Desenvolvimento da Automação

Tecnologias Utilizadas

Tecnologia	Versão	Uso
Python	3.14	Linguagem principal de desenvolvimento
Playwright	1.61.0	Automação do navegador para extração de dados
python-docx	1.2.0	Geração do documento Word (.docx)
smtplib	(stdlib)	Envio de e-mail via protocolo SMTP
python-dotenv	1.2.2	Carregamento seguro de variáveis de ambiente

Extração de Dados (Playwright)

O script `extracao.py` realiza a automação web utilizando a biblioteca Playwright para Python. O fluxo de extração funciona da seguinte forma:

- Carrega as variáveis de ambiente do arquivo `.env` via `load_dotenv()`
- Determina a URL do Portal Fake (variável `PORTAL_FAKE_URL` ou fallback para o arquivo `portal_fake/index.html` incluído no repositório)
- Inicia o navegador Chromium (headless ou com interface gráfica)
- Navega até o Portal Fake e clica no botão **"Novo Cadastro"** (`#btnNovo`)
- Aguarda 1 segundo para o formulário carregar os dados
- Extraí os valores dos 7 campos via seletores CSS:

Campo	Seletor CSS	Chave no dicionário
Nome	<code>#f_nome</code>	<code>nome</code>
Sobrenome	<code>#f_sobrenome</code>	<code>sobrenome</code>
CPF	<code>#f_cpf</code>	<code>cpf</code>

E-mail	#f_email	email
Telefone	#f_telefone	telefone
Nascimento	#f_nascimento	data_nascimento
Endereço	#f_endereco	endereco

1. Retorna os dados como um dicionário Python para uso nos demais módulos

Código Relevante — Extração (extracao.py)

```
def extrair_dados() -> dict: load_dotenv() url_portal = os.getenv("PORTAL_FAKE_URL") if not url_portal: caminho_portal = ( Path(__file__).resolve().parent.parent / "portal_fake" / "index.html" ) url_portal = caminho_portal.resolve().as_posix() if not url_portal.startswith(("http://", "https://", "file://")): url_portal = f"file://{url_portal}" with sync_playwright() as p: navegador = p.chromium.launch(headless=True) pagina = navegador.new_page() pagina.goto(url_portal) pagina.click("#btnNovo") pagina.wait_for_timeout(1000) dados = { "nome": pagina.locator("#f_nome").input_value(), "sobrenome": pagina.locator("#f_sobrenome").input_value(), # ... demais campos } navegador.close() return dados
```

Código Relevante — Orquestrador (main.py)

```
def main(): load_dotenv() # Etapa 1: Extração dados = extrair_dados() # Etapa 2: Geração do documento caminho_ficha = gerar_ficha_cadastro(dados) # Etapa 3: Envio de e-mail enviar_email( destinatario=os.getenv("EMAIL_DESTINATARIO"), assunto=f"Ficha de Cadastro - {dados['nome']} {dados['sobrenome']}", corpo="...", caminho_anexo=caminho_ficha, )
```

5. Documento e Envio de E-mail

Geração da Ficha (.docx)

O módulo `documento_email.py` gera a ficha de cadastro em formato Word utilizando a biblioteca `python-docx`. O documento gerado contém:

- **Título:** "FICHA DE CADASTRO DE CLIENTE" (centralizado, nível H1)
- **Subtítulo:** "Portal Fake Soluções Digitais" (centralizado)
- **Tabela de dados:** 7 linhas × 2 colunas com os dados do cliente (campo em negrito na coluna esquerda, valor na coluna direita)
- **Documentos necessários:** Checklist com ☐ Documento oficial com foto, ☐ Comprovante de residência, ☐ Ficha preenchida e convertida para PDF
- **Declaração:** Texto de veracidade das informações
- **Campo de assinatura:** Linha para assinatura e data

O arquivo é salvo na pasta `resources/` com nome no formato `ficha_cadastro_YYYYMMDD_HHMMSS.docx`.

Exemplo de saída da execução:

```
[2/3] Gerando ficha de cadastro (.docx)... ✓ Ficha gerada:
/mnt/projetos/romulus/HyperAutomation/resources/ficha_cadastro_20260723_133553.docx
```

Envio de E-mail

O módulo `envio_email.py` implementa o envio automatizado de e-mail com as seguintes características:

- **Protocolo:** SMTP com STARTTLS (criptografia em trânsito)
- **Composição:** Mensagem MIME multipart com corpo em texto simples e anexo binário (.docx) codificado em Base64
- **Autenticação:** Login via credenciais carregadas do arquivo `.env`
- **Tratamento de erros:** Exceções de conexão, autenticação e envio são capturadas e reportadas com mensagens claras

```
def enviar_email(destinatario, assunto, corpo, caminho_anexo=None): load_dotenv() remetente
= os.getenv("EMAIL_REMETENTE") senha = os.getenv("EMAIL_SENHA") mensagem = MIMEMultipart()
mensagem["From"] = remetente mensagem["To"] = destinatario mensagem["Subject"] = assunto
mensagem.attach(MIMEText(corpo, "plain", "utf-8")) # Anexa o arquivo .docx with
open(caminho_anexo, "rb") as arquivo: parte = MIMEBase("application", "octet-stream")
parte.set_payload(arquivo.read()) encoders.encode_base64(parte) mensagem.attach(parte) with
smtpplib.SMTP(smtp_host, smtp_port) as servidor: servidor.starttls()
servidor.login(remetente, senha) servidor.sendmail(remetente, destinatario,
mensagem.as_string())
```

Uso do `.env`

As credenciais sensíveis (e-mail, senha, configuração SMTP) são protegidas utilizando o padrão de variáveis de ambiente:

1. O arquivo `.env` armazena as credenciais reais (não versionado)
2. O arquivo `.env.example` documenta as variáveis necessárias (versionado como template)
3. O `.gitignore` inclui `.env` para impedir o commit acidental de credenciais
4. O código utiliza `python-dotenv` (`load_dotenv()`) para carregar as variáveis e `os.getenv()` para acessá-las

```
# .env.example – Template de variáveis EMAIL_REMETENTE=seuemail@gmail.com
EMAIL_SENHA=sua_senha_de_app EMAIL_DESTINATARIO=destinatario@email.com
SMTP_HOST=smtp.gmail.com SMTP_PORT=587
```

6. Versionamento (GitFlow)

Estratégia de Branches

O projeto seguiu a metodologia **GitFlow** com as seguintes branches:

fae5c29	chore: merge release/1.0 para entrega	Romulo Lira	23/07/2026
---------	---	-------------	------------

Publicação no GitHub

- **Repositório:** <https://github.com/CodeMartell/HyperAutomation>
- **Tag:** v1.0 — *Release 1.0 - Entrega Semana06 AX Academy*
- **Branches publicadas:** main, develop, release/1.0, feature/extracao, feature/documento-email

7. Testes e Evidências

7.1 Robô em execução

Saída completa do terminal executando `python source/main.py`:

```
===== HyperAutomation – Portal Fake
Soluções Digitais ===== [1/3] Extraindo
dados do Portal Fake... Dados extraídos: nome: Lucas sobrenome: Ferreira cpf:
135.028.738-49 email: lucas.ferreira@email.com telefone: (43) 960019-2785 data_nascimento:
17/04/1989 endereco: Av. Getúlio Vargas, 1234, Belo Horizonte - MG ✓ Dados extraídos com
sucesso. [2/3] Gerando ficha de cadastro (.docx)... ✓ Ficha gerada:
/mnt/projetos/romulus/HyperAutomation/resources/ficha_cadastro_20260723_134431.docx [3/3]
Enviando e-mail com a ficha em anexo... Conectando ao servidor SMTP (smtp.gmail.com:587)...
E-mail enviado com sucesso para romulolira1@hotmail.com. ✓ E-mail enviado com sucesso.
===== Fluxo concluído com sucesso! Ficha:
/mnt/projetos/romulus/HyperAutomation/resources/ficha_cadastro_20260723_134431.docx E-mail
enviado para: romulolira1@hotmail.com
=====
```

7.2 Dados extraídos

Dados extraídos automaticamente do Portal Fake na execução de teste:

Campo	Valor Extraído
Nome	Lucas
Sobrenome	Ferreira
CPF	135.028.738-49
E-mail	lucas.ferreira@email.com
Telefone	(43) 960019-2785

Data de Nascimento	17/04/1989
Endereço	Av. Getúlio Vargas, 1234, Belo Horizonte - MG

7.3 Ficha de cadastro gerada

Arquivo gerado: `resources/ficha_cadastro_20260723_134431.docx` (37 KB)

O documento Word contém: - Título centralizado "FICHA DE CADASTRO DE CLIENTE" - Subtítulo "Portal Fake Soluções Digitais" - Tabela com os 7 campos de dados do cliente - Seção de documentos necessários com checklist - Declaração de veracidade e campo de assinatura

7.4 E-mail recebido

E-mail enviado com sucesso via servidor SMTP (`smtp.gmail.com:587`) para a caixa de destino `romulolira1@hotmail.com`, contendo em anexo a ficha de cadastro gerada `ficha_cadastro_20260723_134431.docx`.

7.5 Repositório no GitHub

URL: <https://github.com/CodeMartell/HyperAutomation>

Estrutura de branches no repositório:

```
main ← release/1.0 ← develop ← feature/extracao ← feature/documento-email Tag: v1.0
(Release 1.0 - Entrega Semana06 AX Academy)
```

Arquivos na branch main (14 arquivos):

```
.env.example .gitignore README.md images/ficha_portal_fake_sd_bpmn.drawio
/mnt/projetos/romulus/HyperAutomation/images/ficha_portal_fake_sd_bpmn.png
portal_fake/index.html relatorio_tecnico_template.md requirements.txt
resources/ficha_cadastro_20260721_165738.docx source/documento_email.py
source/envio_email.py source/extracao.py source/main.py
```

8. Conclusão

Dificuldades Encontradas

Caminho do Portal Fake: O Portal Fake original era um arquivo HTML fornecido pelo professor e referenciado por caminho absoluto Windows (`C:/Users/Turma02/...`), o que impossibilitava a execução em outros ambientes. A solução foi recriar o portal e incluí-lo no repositório, com configuração via variável de ambiente para flexibilidade.

Autenticação SMTP: O Gmail exige uma "Senha de App" (App Password) para autenticação via `smtplib`, o que requer ativação da verificação em duas etapas na conta Google. Essa configuração adicional não era intuitiva e demandou pesquisa.

Integração dos módulos: Inicialmente os scripts de extração e geração de documento eram independentes e não se comunicavam. Foi necessário refatorar o `extracao.py` para retornar os dados extraídos e criar um script orquestrador (`main.py`) para integrar o fluxo completo.

GitFlow: A gestão de branches com GitFlow exigiu atenção especial para garantir que os merges seguissem a ordem correta (feature → develop → release → main) e que a tag fosse criada no momento adequado.

Melhorias Futuras

- **Interface gráfica:** Desenvolver uma GUI (ex.: Tkinter ou web) para facilitar a configuração e execução da automação
- **Suporte a múltiplos portais:** Parametrizar o robô para funcionar com diferentes formulários web, não apenas o Portal Fake
- **Geração de PDF:** Adicionar conversão automática do .docx para PDF
- **Agendamento:** Implementar execução agendada (cron) para processar cadastros em lote periodicamente
- **Logging:** Substituir os prints por um sistema de logging estruturado com arquivo de log
- **Testes automatizados:** Criar testes unitários e de integração para garantir a qualidade do código

Considerações Finais

O projeto HyperAutomation proporcionou uma experiência prática e completa em automação de processos. Ao integrar ferramentas de automação web (Playwright), geração de documentos (python-docx) e comunicação automatizada (smtplib), foi possível vivenciar o ciclo completo de um projeto de Hyperautomation — desde a modelagem do processo (BPMN) até a implementação, teste e publicação.

O uso de boas práticas como GitFlow para versionamento, `.env` para proteção de credenciais e modularização do código em scripts independentes reforçou conceitos importantes de engenharia de software que serão aplicáveis em projetos futuros.